



Bahir Dar University
Bahir Dar Institute of Technology (BiT)
Faculty of Computing
Software Engineering
Internship Report

Hosting Company: Askuala Link Trading PLC

Project Title: Internship Report at Askuala link

By: Ayalnesh Worku [1404571]

Ehitemusie Nebyu [1404565]

Wubamlak Girum [1404373]

Company Supervisor: Mr. Fanuel

Advisor: Mr. Derejaw & Mr. Sertse & Ms. Mieraf

Date: September 24, 2025

Declaration

We confirm that the contents of this internship report accurately reflect the work we carried out during our placement at **Askuala Link** from **March 04, 2025, to June 07, 2025**. The report provides an honest account of the assignments, projects, and responsibilities we were engaged in throughout the internship.

The analysis, documentation, and reflections presented here were prepared independently by us, based on the skills and experiences we gained during the program. Any external sources or materials that have been consulted are properly acknowledged within the report.

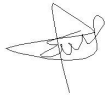
We further affirm that this report is our original work and has not been submitted, either in part or in full, to meet the requirements of any other course or qualification.

Intern name

1. Ayalnesh worku



2. Ehitemusie Nebyu



3. Wubamlak Grum



Advisor name

1. Mr. Derejaw



2. Mr. Sertse



3. Ms. Mieraf



Acknowledgement

We would like to express our sincere gratitude to all those who supported us throughout our internship at **Askuala Link**.

First, we extend our heartfelt appreciation to our mentor, **Fanuel Almaw**, for his continuous guidance, support, and expertise. His mentorship provided us with valuable insights into the professional software development process and helped us navigate the challenges we encountered.

We are also deeply grateful to our university advisors, **Mr. Derejaw, Mr. Sertse, and Ms. Mieraf**, for their supervision and constructive feedback during the preparation of this report. Their guidance ensured that the document meets the required academic standards.

Special thanks also go to the entire **Software Development Team** at Askuala Link for creating a welcoming environment and sharing their expertise with us. Their willingness to answer our questions and involve us in real tasks provided us with hands-on knowledge that complemented our academic learning.

Finally, we would like to thank our families and friends for their constant encouragement and support throughout this journey.

Executive Summary

This report highlights our experiences as software engineering interns at **Askuala Link** from **March 04, 2025, to June 07, 2025**. The company offers a suite of integrated digital tools, including SMS Reporting, AI Suggestions, and Student Performance Tracking, designed to streamline school administration and strengthen collaboration between educators, parents, and students.

During our internship, we were part of a flexible project team focused on both frontend and backend development. Our primary tasks included translating Figma designs into code using Next.js and Flutter, building RESTful APIs with Node.js, and integrating databases. We actively participated in code reviews and collaborated closely to ensure project goals were met.

This experience significantly enhanced our technical skills, making us proficient in full-stack versatility, rapid prototyping, and API design. Beyond technical abilities, we also gained a deeper understanding of agile methodologies, improved our teamwork and communication skills, and developed an entrepreneurial mindset.

Overall, this internship was a transformative experience that bridged the gap between academic knowledge and professional practice. It provided a challenging and rewarding environment that prepared us for the demands of the modern tech industry and offered valuable insights for our future careers.

Catalog

Declaration	2
Acknowledgement	2
Executive Summary	3
Part Two – Company Background	6
2.1 Brief History of Askuala Link	6
2.2 Main Products and Services	7
2.3 Askuala Link’s Main Customers or End Users	8
2.4 Organizational Structure	9
2.5 Workflow in Each Department or Functional Unit	11
Part Three –Internship Experience	13
3.1 Department Placement and Rationale	13
3.2 Executed Work Tasks	13
3.3 Utilized Engineering Methods, Tools, and Techniques	15
3.4 Encountered Challenges and Identified Problems	16
3.5 Implemented Measures to Overcome Challenges and Problems	17
3.6 Enhancements in Practical Skills	18
3.7 Enhancements in Theoretical Knowledge	19
3.8 Strengthened Team Collaboration Skills	20
3.9 Improving Leadership Skills	21
3.10 Understanding Work Ethics and Industrial Psychology	21
3.11 Gaining Entrepreneurship Skills	22
3.12 Enhancing Interpersonal Communication Skills	23
3.13 Recommendations and Conclusion on Our Internship Experience	23
Part Four: Project Work	24
4.1 Summary of the Project	24
4.2 Problem Statement & Justification	31
4.3 Objectives of the Project	35
4.4 Methodology	37
4.5 Analysis, Results, and Discussion	38
4.6 Solution Proposed	40
4.7 Conclusion and Recommendations	42
Part Five: General Conclusion and Recommendations	44
References	46
Appendix	46

Part Two – Company Background

2.1 Brief History of Askuala Link

Askuala Link was established in 2022 in Bahir Dar with the vision of reshaping how schools across Africa communicate and manage information. From its earliest planning stages, the company set out to solve a pressing challenge in education: the gap between administrators, teachers, parents, and students when it comes to reliable, real-time sharing of academic data. Drawing on modern cloud technologies, the founders built a platform specifically tailored for the diverse needs of African schools, where many institutions still rely on paper records and fragmented communication methods. The goal was clear—create a single, intelligent system that streamlines school management while empowering every participant in the learning process.

During its first year of operation, Askuala Link focused on developing a secure and scalable digital environment. The team designed core services such as real-time communication channels and robust academic data management, ensuring that attendance records, grades, and performance histories could be shared instantly and safely. By building both a web interface and mobile applications, the company made it possible for parents and teachers to stay connected whether they were in urban centers or rural areas with limited resources. This early commitment to accessibility became a defining characteristic of the brand.

As the platform matured, Askuala Link expanded beyond basic school management to introduce innovative, AI-driven features that set it apart in the Ethiopian EdTech landscape. The company launched tools like automated SMS reporting, AI-powered suggestions to help parents support their children, and intelligent roster management for administrators. These features were developed in close collaboration with educators and school leaders, ensuring that each addition directly addressed the everyday challenges of running a school. The emphasis on artificial intelligence not only modernized classroom administration but also opened new possibilities for personalized learning.

Partnerships quickly became a cornerstone of the company's growth. Askuala Link built strong relationships with respected local and international organizations that share its mission of transforming education. Through these collaborations, the company gained access to mentorship, funding opportunities, and advanced technological resources. Such alliances accelerated product development and enabled the platform to scale to more schools and regions, reinforcing its reputation as a trusted partner in the educational ecosystem.

Today, Askuala Link stands as a dynamic example of how Ethiopian innovation can drive continental change. With a steadily growing team and an expanding customer base, the company continues to refine its services to meet the evolving demands of schools of all sizes. Its mission to revolutionize education in Africa through AI-powered tools guides every decision, from enhancing user experience to maintaining rigorous data security. In just a few years, Askuala Link has progressed from a start-up idea to a leading force in digital education management, proving that technology built with local insight can have a lasting impact on how students learn and succeed.

2.2 Main Products and Services

Askuala Link offers a rich set of integrated digital tools that modernize school administration and strengthen collaboration between educators, parents, and students. At the core of its platform is **SMS Reporting**, a feature designed to keep parents constantly informed about their child's academic life. By automating attendance and grade updates, the system removes repetitive manual work for teachers while delivering timely, accurate information to families. Schools can configure the service to send updates at chosen intervals and in multiple languages, ensuring that parents remain engaged regardless of schedule or location.

The company's **AI Suggestions** service brings advanced artificial intelligence to the heart of education. Drawing on each student's performance data, the system generates personalized insights and practical recommendations for parents. These might include targeted study tips, subject-specific resources, or alerts about areas needing improvement. Because the AI adapts continuously to new information, the guidance evolves with the student's progress, offering a level of individualized support that traditional reporting cannot match.

Efficient administration is further enhanced through **Roster Management**, which simplifies the creation and maintenance of school rosters. Administrators can add, remove, or update student and teacher records with minimal effort, while real-time updates keep information accurate across the platform. This capability is particularly valuable for schools managing large numbers of students and frequent enrollment changes, reducing the chance of errors and ensuring smooth day-to-day operations.

Another cornerstone of the platform is **Student Performance Tracking**. This feature gives teachers a detailed, up-to-the-minute view of each learner's academic journey, enabling them to spot trends, identify difficulties early, and coordinate timely interventions with parents. The transparency

created by real-time data ensures that teachers, parents, and students share a common understanding of goals and progress, ultimately leading to stronger outcomes.

Recognizing the importance of teacher engagement, Askuala Link includes a **Teacher Bonuses** module. Schools can reward educators for contributions such as accurate data entry and administrative support, reinforcing a culture of accountability and appreciation. By motivating staff and highlighting their efforts, this feature not only improves morale but also enhances the overall quality of school data management.

Finally, the platform strengthens the vital relationship between home and school through **Parent–Teacher Communication** tools. A dedicated messaging channel allows parents to contact teachers directly with questions or concerns and receive prompt, organized responses. This structured dialogue replaces scattered phone calls and paper notes, ensuring that communication is clear, consistent, and productive.

Together, these services create a seamless ecosystem where information flows freely, decisions are data-driven, and every participant administrator, teacher, parent, and student—can focus on what matters most: meaningful teaching and learning.

2.3 Askuala Link’s Main Customers or End Users

Since its launch, **Askuala Link** has focused on schools and educational organizations that want to modernize the way they manage data and communicate with parents and students. Headquartered in Bahir Dar, the company now serves a growing number of private and public schools across Ethiopia and is steadily expanding its reach to other regions of the country. Its goal is to give schools of all sizes a reliable, secure, and easy-to-use platform that improves communication and simplifies daily operations.

Askuala Link’s **primary customers** are school owners, principals, and administrative teams. These decision makers adopt the platform to replace paper records and scattered phone calls with a single, cloud-based system. They use it to manage student rosters, automate attendance reporting, and generate accurate performance summaries, reducing administrative work and ensuring that critical information is always up to date.

The company also serves **teachers**, who are among the most active users of the platform. Teachers record grades, track attendance, and monitor student progress in real time. They benefit from tools such as AI Suggestions and Student Performance Tracking, which help them identify learners who

need extra support and share targeted feedback with parents. Features like Teacher Bonuses recognize their efforts and encourage accurate, timely data entry.

Parents and guardians form another key group of end users. Through automated SMS updates and a dedicated messaging channel, parents stay informed about attendance, grades, and important announcements. They receive personalized recommendations from the AI system, allowing them to support their children's learning at home. By keeping families engaged and well informed, the platform strengthens the partnership between home and school.

Finally, **students themselves** gain direct advantages from the system. They can view their own academic records, follow progress reports, and respond quickly to feedback from teachers and parents. Real-time visibility of their performance helps them take responsibility for their learning and stay motivated.

Askuala Link currently serves a **growing number of private and public schools across Ethiopia**, including well-established institutions in **Bahir Dar, Gonder, and other major cities**. Through these connections—school administrators, teachers, parents, and students—the company creates a single digital environment where every stakeholder benefits. By offering reliable communication tools, secure data management, and intelligent insights, Askuala Link meets the needs of modern education and supports continuous improvement in schools throughout the country.

2.4 Organizational Structure

Askuala Link appears to have a relatively flat and functional organizational structure, typical of a technology startup. The leadership team is at the top, overseeing specialized functional units or teams that handle specific aspects of the business, from technical development to customer relations.

Leadership

The company is led by key executives who manage the overall strategy and operations. The most prominent roles identified are:

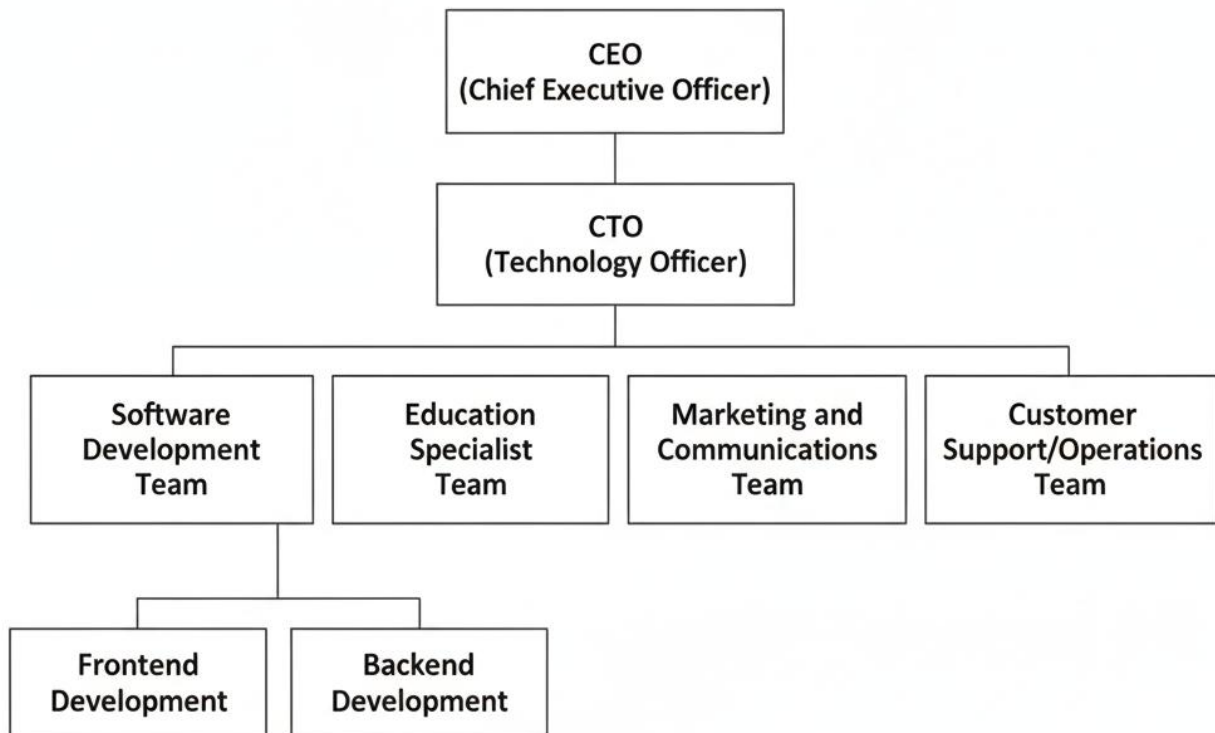
- **CEO (Chief Executive Officer):** Responsible for the company's vision and business strategy.
- **CTO (Chief Technology Officer):** Oversees all technology and development-related activities, ensuring the platform remains at the forefront of the industry.

Functional Teams

Under the leadership, the company's work is organized into specialized teams, each focused on a specific function. Based on the career opportunities and services offered, these teams likely include:

- **Software Development Team:** This is the core of the company, responsible for building and maintaining the AI-powered school management system. This team is likely divided into a **Frontend Development** unit (for user interfaces) and a **Backend Development** unit (for system and server logic).
- **Education Specialist Team:** This team collaborates to create meaningful educational content and innovative solutions that are integrated into the platform.
- **Marketing and Communications Team:** This team is responsible for promoting Askuala's mission, engaging with the user community, and managing public relations.
- **Customer Support/Operations Team:** While not explicitly named, the company's focus on user support and service suggests a team dedicated to assisting schools, parents, and students, and managing day-to-day operations.

Askula Link Organizational Structure



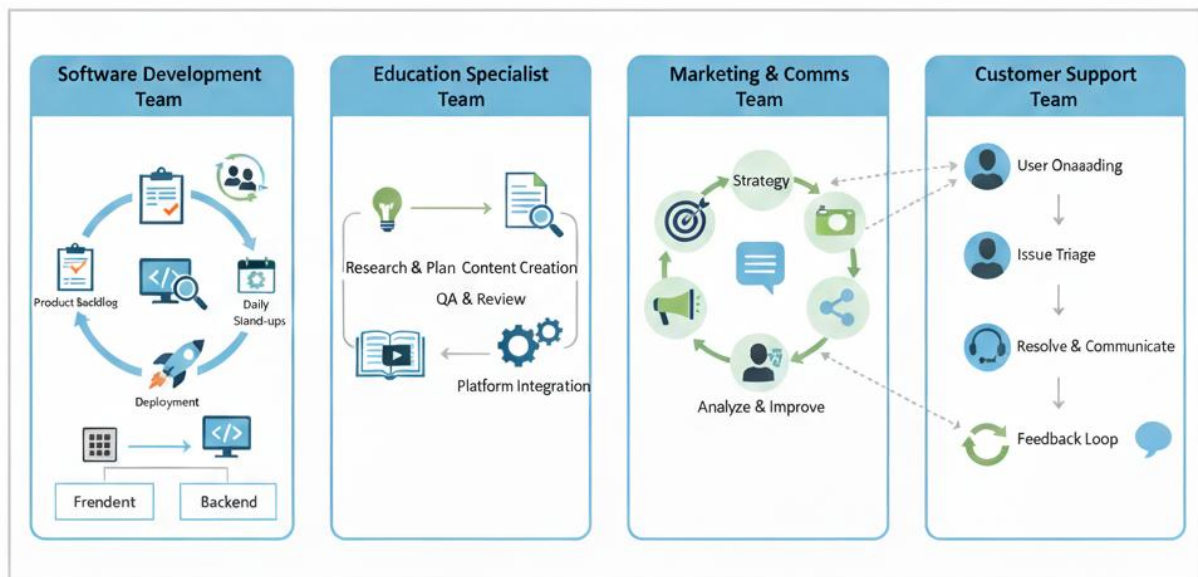
2.5 Workflow in Each Department or Functional Unit

Askula Link organizes its operations around a project-focused approach, utilizing agile practices to ensure flexibility and efficiency. Projects are broken into iterative cycles, allowing teams to plan, develop, test, and deliver solutions in manageable stages. Daily team check-ins help monitor progress, resolve challenges, and maintain alignment with project goals.

Each project team is composed of software developers, system engineers, designers, and testers, led by a project manager or team lead. Developers collaborate closely with designers and education specialists to translate client requirements and educational objectives into functional software solutions, while junior staff work alongside senior engineers to gain experience and adopt best practices.

Testing is integrated throughout the work-flow, with both automated and manual tests conducted regularly to identify issues early and maintain high-quality standards. Project managers coordinate the team's tasks, track progress, and facilitate communication across all functional units. They maintain product backlogs, review sprint outcomes, and ensure deliverables meet client expectations.

This structured yet flexible work-flow enables Askuala Link teams to respond quickly to changes, encourage collaboration, and ensure accountability at every stage of development. By focusing on client-specific projects, the company maintains consistent quality while supporting continuous improvement and skill development among team members.



Part Three –Internship Experience

3.1 Department Placement and Rationale

During our internship, our team of three was placed in a dynamic and flexible project team. Unlike a rigid structure, we were organized around project requirements rather than fixed roles. This allowed us to function as a versatile unit, with each of us contributing to both frontend and backend development. This setup was particularly beneficial as it exposed us to a wide range of technologies, from Next.js for web frontend to Flutter for mobile applications, and Node.js and Django for backend services. One of our teammates, Wubamlak, gravitated more towards backend development, primarily focusing on Node.js.

The rationale behind this flexible placement was to prepare us for the agile and ever-changing nature of the tech industry. By not being confined to a single technology or role, we learned to adapt quickly, prioritize tasks based on project needs, and become full-stack developers in a true sense. This arrangement also taught us how to collaborate effectively on various parts of a project, fostering strong communication and teamwork skills. We learned to support each other and take on any task required to get the job done, whether it was frontend or backend.

3.2 Executed Work Tasks

Our work was driven by client requirements, which meant our tasks were diverse and often required us to learn new tools on the fly. Here's a breakdown of what we accomplished:

- **Translating Figma Designs to Code:** A primary task for Ehitmusie and Ayalnesh was to convert client-provided Figma designs into functional code. We used Next.js for web projects and Flutter for mobile apps, focusing on creating pixel-perfect and responsive user interfaces.
- **Building RESTful APIs:** Wubamlak took the lead on most of the backend tasks, designing and implementing RESTful APIs using Node.js. These APIs served as the communication layer between our frontend applications and the database. The APIs included user authentication systems using JWT tokens,

CRUD operations for data management, file upload and storage functionality, integration with external services like email and SMS providers, real-time features using WebSockets, and security measures like rate limiting and input validation.

- **Database Integration:** We were responsible for integrating our applications with various databases. Wubamlak, in particular, handled database schema design and management, ensuring efficient data storage and retrieval. This included designing normalized database schemas, implementing foreign key relationships, writing complex SQL queries for data analysis, creating database migration scripts, optimizing query performance through indexing, and implementing data backup strategies.
- **Frontend-Backend Integration:** Ehitemusie and Ayalnesh worked on connecting the frontend to the backend APIs. This involved writing API request functions with proper error handling, implementing loading states and user feedback mechanisms, managing application state based on server responses, handling different response formats and error codes, implementing offline functionality and data synchronization, and creating reusable API service modules.
- **Code Reviews and Collaboration:** We actively participated in code reviews, providing feedback on each other's work and ensuring code quality. This process was essential for maintaining consistency and learning from each other's coding styles and practices.
- **Adopting New Frameworks:** We were often tasked with building projects in frameworks we were not familiar with. This required us to quickly read documentation, watch tutorials, and apply our foundational knowledge to new environments.
- **Client Feedback Implementation:** We frequently had to revise our work based on client feedback. This sometimes meant refactoring large sections of code or even changing the entire project's framework if the client's needs changed, a challenging but crucial part of professional development.

3.3 Utilized Engineering Methods, Tools, and Techniques

Our flexible approach required us to be proficient with a wide array of tools and technologies.

- **Version Control:** Git was our primary tool for managing code. We used it extensively to branch for new features, merge our work, and resolve conflicts. GitHub was our central repository for project management. We implemented feature branching strategies, pull request processes with code reviews, conflict resolution techniques, backup and disaster recovery procedures, and integration with project management tools.
- **Design and Prototyping:** We used Figma to access and understand the design requirements. This tool was our single source of truth for UI/UX specifications. We learned to extract precise measurements and colors, collaborate with designers on iterations, create interactive prototypes, and maintain design consistency across projects.
- **Development Environment:** We used Visual Studio Code as our main code editor due to its versatility and extensive plugin ecosystem, which supported all the languages and frameworks we worked with. We customized it with extensions, integrated with Git and terminal tools, and configured linting and formatting tools.
- **Communication:** We relied on platforms like Slack and telegram for quick communication. We organized project-specific channels, shared code snippets and debugging information, conducted virtual meetings and screen sharing, managed file sharing and collaboration, and set up custom bots for project automation.
- **Backend and Database Tools:** We used a variety of tools depending on the project. We used Postman to test APIs, while we utilized different database management tools for MySQL and PostgreSQL. We created comprehensive API test suites, set up automated testing workflows, documented API endpoints with examples, tested different authentication methods, and performed load testing for critical endpoints.

3.4 Encountered Challenges and Identified Problems

The flexible nature of our internship, while rewarding, presented unique challenges that forced us to grow.

- **Frequent Framework and Language Changes:** The biggest challenge was the constant need to switch between frameworks and languages. We would be working on a Node.js project one week and a Django project the next. This made it difficult to master any single technology, build muscle memory for specific syntax, develop deep expertise in framework-specific best practices, maintain consistent coding standards across projects, and remember the specific quirks of each framework.
- **Complete Project Rewrites:** We faced the disheartening task of having to redo entire projects. If a client decided to change the technology stack or their core requirements, we had to start from scratch. This was a valuable lesson in managing client expectations and adapting to change. We learned about the importance of flexible architecture, the need for clear client communication, the value of rapid prototyping, and the reality that technical decisions have long-term consequences.
- **Learning on the Job:** We were often given tasks that required us to use frameworks we were completely unfamiliar with. This meant that a significant portion of our time was spent learning and understanding new concepts and documentation, often under tight deadlines. We had to make critical decisions with limited knowledge, debug issues in unfamiliar codebases, implement complex features without full understanding, and make mistakes that could have been avoided with more time.
- **Managing Client Feedback:** Receiving last-minute or significant changes in client feedback was difficult. It required not only technical skill but also the ability to communicate effectively and manage project scope. We learned to handle unrealistic expectations, manage scope creep, communicate the impact of changes, negotiate reasonable timelines, and maintain professional relationships during difficult conversations.

- **Context Switching Overhead:** Working on multiple projects simultaneously created significant overhead. We had to constantly switch between different codebases, remember different project requirements, manage different deadlines, coordinate with different clients, and maintain focus across competing priorities.
- **Imposter Syndrome:** The constant need to learn new technologies and deliver results under pressure led to feelings of inadequacy. We struggled with self-doubt, anxiety about making mistakes, fear of asking questions, pressure to appear competent, and difficulty maintaining confidence during challenging periods.

3.5 Implemented Measures to Overcome Challenges and Problems

We developed several strategies to tackle these challenges and ensure project success.

- **Aggressive Research and Learning:** We became proficient at quickly researching and learning new technologies. We relied heavily on official documentation, online tutorials, and community forums. We developed structured learning approaches, created personal documentation and cheat sheets, and shared learning resources with team members.
- **Enhanced Team Communication:** We held daily stand-up meetings to discuss project progress and any roadblocks we were facing. This open communication helped us share knowledge and coordinate our efforts, especially when switching between different technologies. We established regular check-ins, shared progress updates, coordinated work across projects, identified potential risks early, and planned tasks collaboratively.
- **Focusing on Foundational Concepts:** Instead of getting lost in the syntax of a new framework, we focused on understanding the underlying architectural patterns and principles. This allowed us to apply our core knowledge to new tools more easily. We studied universal programming principles, learned

common design patterns, understood database design concepts, and applied security best practices across all technologies.

- **Collaborative Problem-Solving:** When faced with a difficult bug or a new technology, we worked together to solve it. We would pair-program or brainstorm solutions as a team, leveraging our collective knowledge to overcome challenges.

3.6 Enhancements in Practical Skills

This internship, with its diverse technical demands, significantly enhanced our practical skills.

- **Full-Stack Versatility:** We are now comfortable working on both the frontend and backend of an application, regardless of the technology stack.
- **Rapid Prototyping and Development:** We became adept at quickly building prototypes and applications from a given design, a critical skill in a fast-paced environment. We learned to create working prototypes quickly, iterate based on feedback, build minimum viable products, test concepts before full implementation, and present ideas through demos.
- **Technical Adaptability:** We developed the ability to quickly pick up new languages, frameworks, and libraries, a skill that is highly valued in the modern tech industry.
- **API Design and Consumption:** We gained extensive experience in designing robust APIs and integrating them with various frontend applications, ensuring smooth data flow.
- **Debugging and Troubleshooting:** Our frequent encounters with bugs across different technology stacks strengthened our debugging skills. We learned to debug across multiple layers, use appropriate debugging tools, analyze error logs and stack traces, systematically isolate problems, and prevent similar issues in the future.
- **Database Management:** We gained comprehensive experience with different database systems. We learned to design normalized schemas, write complex

queries, optimize performance, implement migrations, and manage data integrity.

- **Version Control:** We became proficient with Git and GitHub workflows. We learned advanced branching strategies, conflict resolution, pull request processes, automated testing integration, and collaborative development practices.

3.7 Enhancements in Theoretical Knowledge

Beyond the practical skills, the internship deepened our theoretical understanding of software development.

- **Architectural Patterns:** We gained a better understanding of different architectural patterns, such as the MVC pattern in Django and the component-based architecture of React and Next.js. We learned to recognize and implement common patterns, understand separation of concerns, apply modular design principles, and maintain consistent architecture across projects.
- **Software Design Principles:** We learned about principles like modularity, separation of concerns, and code reusability from our experience with various frameworks. We studied SOLID principles, learned about design patterns, understood the importance of clean code, and applied best practices consistently.
- **Database Management:** We gained theoretical knowledge of database design, normalization, and query optimization, which was crucial for building scalable backend systems. We learned about relational database theory, understood normalization principles, studied query optimization techniques, and gained knowledge of different database types.
- **Agile Methodologies:** We experienced the benefits of an agile workflow firsthand, learning to adapt to changing requirements and prioritize tasks effectively. We practiced sprint planning, daily stand-ups, retrospectives, user story creation, and continuous improvement.

- **Client Management and Communication:** We learned about the theory behind managing client relationships, including the importance of clear communication, setting expectations, and managing project scope. We studied stakeholder management, communication strategies, expectation management, conflict resolution, and relationship building.

3.8 Strengthened Team Collaboration Skills

Our flexible project setup was a crucible for developing strong teamwork and communication skills.

- **Alignment and Coordination:** We learned to constantly align our work, ensuring that our frontend and backend efforts were synchronized, even when we were using different technologies.
- **Flexible Roles:** The fluidity of our roles meant we had to trust each other and be flexible. We learned to step in and help with any task, fostering a "get it done" attitude. We developed adaptive leadership skills, learned to support each other, and maintained team morale during difficult periods.
- **Constructive Feedback:** We became adept at giving and receiving constructive feedback, which was essential for our rapid learning process and for maintaining code quality across different frameworks. We learned to provide specific, actionable feedback, balance criticism with recognition, focus on behavior and results, and create a safe environment for honest communication.
- **Shared Responsibility:** We developed a strong sense of shared responsibility. If one of us was struggling with a new framework, the others would jump in to help, understanding that the project's success was a collective goal.

3.9 Improving Leadership Skills

The unstructured environment provided us with numerous opportunities to develop leadership skills.

- **Taking Initiative:** We all had to take the initiative to learn new technologies and propose solutions, which helped us develop a proactive mindset. We learned to identify opportunities for improvement, take action without being asked, share knowledge and resources, and lead by example.
- **Mentoring and Knowledge Sharing:** We regularly mentored each other, sharing our knowledge of different frameworks and technologies. For example, Wubamlak took the lead on Node.js and helped us understand backend concepts, while we guided him on frontend best practices. We learned to break down complex concepts, provide hands-on guidance, create learning materials, and adapt teaching methods to different learning styles.
- **Problem-Solving Leadership:** When faced with a difficult challenge, we would often take the lead in researching solutions and coordinating the team's efforts to solve it. We learned to identify problems early, research solutions systematically, coordinate team efforts, take responsibility for outcomes, and learn from failures.

3.10 Understanding Work Ethics and Industrial Psychology

Our internship gave us a unique perspective on the professional world, particularly on the psychological aspects of working in a dynamic environment.

- **Resilience and Adaptability:** We learned that a core part of a developer's job is not just coding, but also adapting to constant change. The frequent rewrites and framework changes taught us resilience and flexibility.

- **Managing Frustration:** We learned to handle the frustration that comes with technical challenges and client feedback. This experience taught us to be patient, systematic, and to see setbacks as learning opportunities.
- **Motivation and Self-Direction:** With no fixed roles, we were responsible for motivating ourselves and directing our own work. This helped us develop strong self-discipline and an entrepreneurial mindset.
- **Communication in a Crisis:** The moments when we had to redo a project were tough, but they taught us how to communicate effectively under pressure.

3.11 Gaining Entrepreneurship Skills

Our flexible, project-based internship gave us a crash course in an entrepreneurial mindset. We learned to think beyond our immediate tasks and approach our work with a focus on innovation and initiative.

- **Identifying Opportunities:** We honed our ability to spot opportunities for innovation and growth. This meant more than just completing a task; it involved looking for ways to improve the product, streamline our workflow, or suggest new features.
- **Problem-Solving and Initiative:** The dynamic nature of our work, which often included changing frameworks and client demands, forced us to become proactive problem-solvers. We couldn't wait for instructions; we had to take the initiative to research solutions and propose new approaches. This developed a forward-thinking, proactive mindset essential for any aspiring entrepreneur or leader.
- **Networking:** We learned the value of building professional networks. Connecting with senior developers and other interns gave us a new perspective on the industry. We saw how building relationships could lead to future opportunities, collaborations, and valuable insights that you can't get from a textbook.

3.12 Enhancing Interpersonal Communication Skills

Working in a close-knit team required us to sharpen our communication skills. We quickly learned that great ideas mean nothing if you can't communicate them effectively.

- **Active Listening and Clear Communication:** We made a conscious effort to actively listen to each other's ideas and feedback, ensuring we understood different viewpoints before responding. At the same time, we practiced conveying our own thoughts clearly and succinctly, which was especially important when explaining complex technical concepts.
- **Collaboration and Teamwork:** Our internship was a masterclass in collaboration. We learned to work effectively with each other, leveraging our different strengths to tackle a variety of projects. This experience taught us how to build rapport and foster a positive, supportive team atmosphere where everyone's contribution was valued.
- **Constructive Feedback:** We developed the ability to both give and receive constructive feedback. This was crucial for our rapid growth, as we were constantly learning from each other's code and problem-solving approaches. We learned to view feedback not as criticism but as a tool for improvement.
- **Conflict Resolution:** When disagreements arose, we learned to handle them maturely and respectfully. We focused on finding common ground and addressing issues directly, which helped us navigate disagreements without compromising team harmony.

3.13 Recommendations and Conclusion on Our Internship Experience

Our internship was a transformative experience that bridged the gap between academic knowledge and real-world professional practice. It provided a flexible, challenging, and incredibly rewarding environment that prepared us for the demands of the modern tech industry.

We strongly recommend that all students seek out internship opportunities. They are essential for gaining hands-on experience, building a professional network, and developing critical skills that can't be taught in a classroom. Our experience taught us that internships are more than just a line on a resume; they are a chance to gain valuable insights into industry dynamics, refine career goals, and cultivate a sense of professionalism and accountability.

The practical skills, theoretical knowledge, and personal growth we achieved have given us a strong foundation for our future careers. We are grateful for the opportunities and guidance we received and believe that the lessons we learned will continue to guide us toward success. We encourage others to embrace similar opportunities to expand their perspectives and realize their full potential.

This internship has not only prepared us for our immediate career goals but has also instilled in us the mindset and skills needed for long-term success in the dynamic and ever-evolving field of software development. We are excited to apply what we have learned and continue growing as professionals in this exciting industry.

Part Four: Project Work

4.1 Summary of the Project

The project that we came up with is called "Software Company System," a comprehensive computer-based system that seeks to simplify organizational processes, improve employee communication, and provide services to community members at speed and efficiency. The overall goal of this project is to create an integrated system whereby administrators, agents, partners, assistants, and customers can communicate seamlessly, maximally boosting productivity and delivery of services at all organizational levels.

A special strength of the Software Company System is that it is highly flexible. It is designed for any software company regardless of its internal organization. If a company has agents, it can utilize the agent functionality; otherwise, it can eliminate it. If a company collaborates with partners, it can activate the partner functionality; if

it doesn't, it can exclude it. Similarly, organizations with administration levels of woreda, zonal, and regional are capable of utilizing the respective features to the full, while others without the similar structures can exclude them. In either case, all organizations can use the universal features—posting news, events, services, and jobs, and receiving customer messages and feedback. This module-based architecture allows the system to develop based on the particular operational needs of each entity.

The general aim of the Software Company System project is to have a computerized web-based portal that integrates all the feasible roles—system administrators, regional, zonal, woreda administrators, assistants, agents, working partners, and customers—into a coordinated workflow. By making it possible for each role to perform its tasks online, whether service monitoring, reporting, requesting services, or ordering services, the platform saves time, increases accountability, and makes timely interventions toward community and organizational needs.

figure 4.1.1 (Admin side of the website).

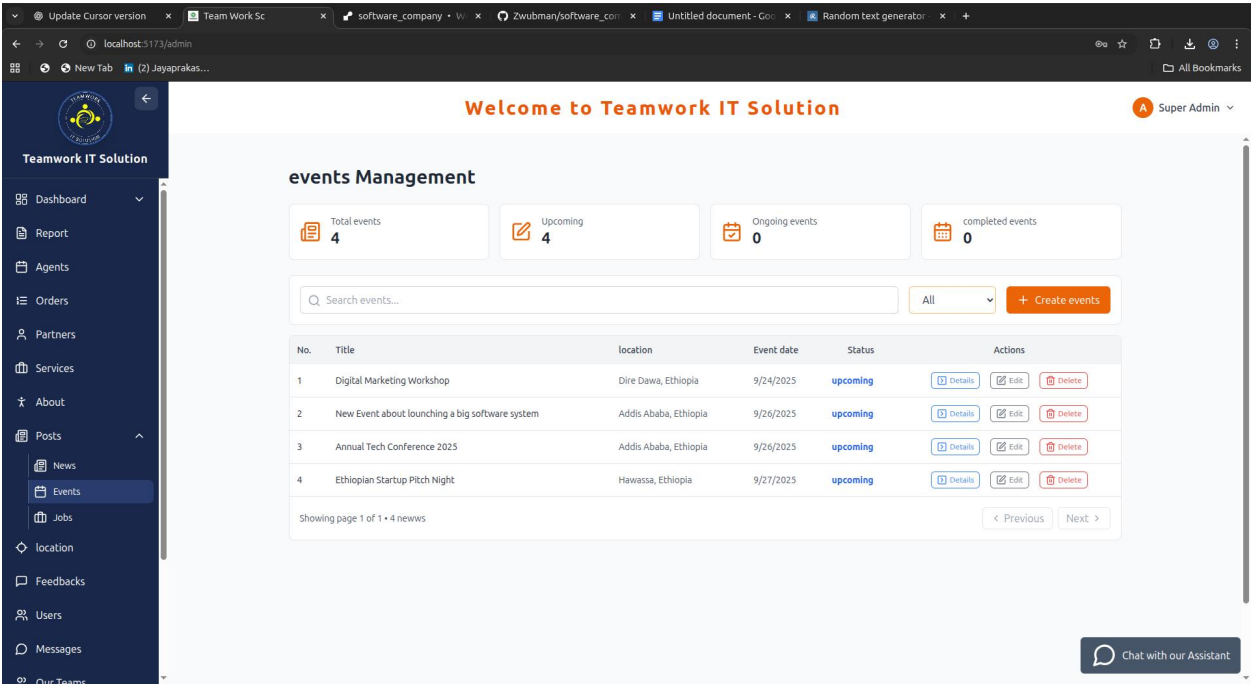


Figure 4.1.1.2 Event management page

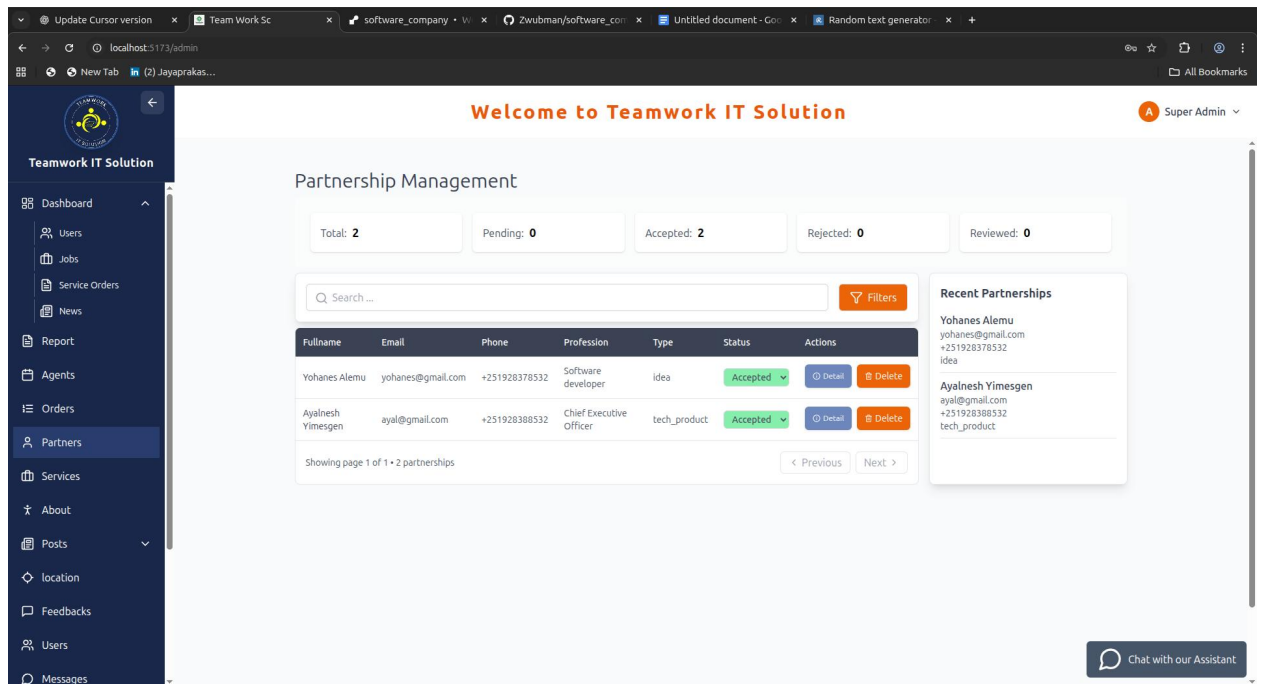


Figure 4.1.1.2 Partnership requests management page

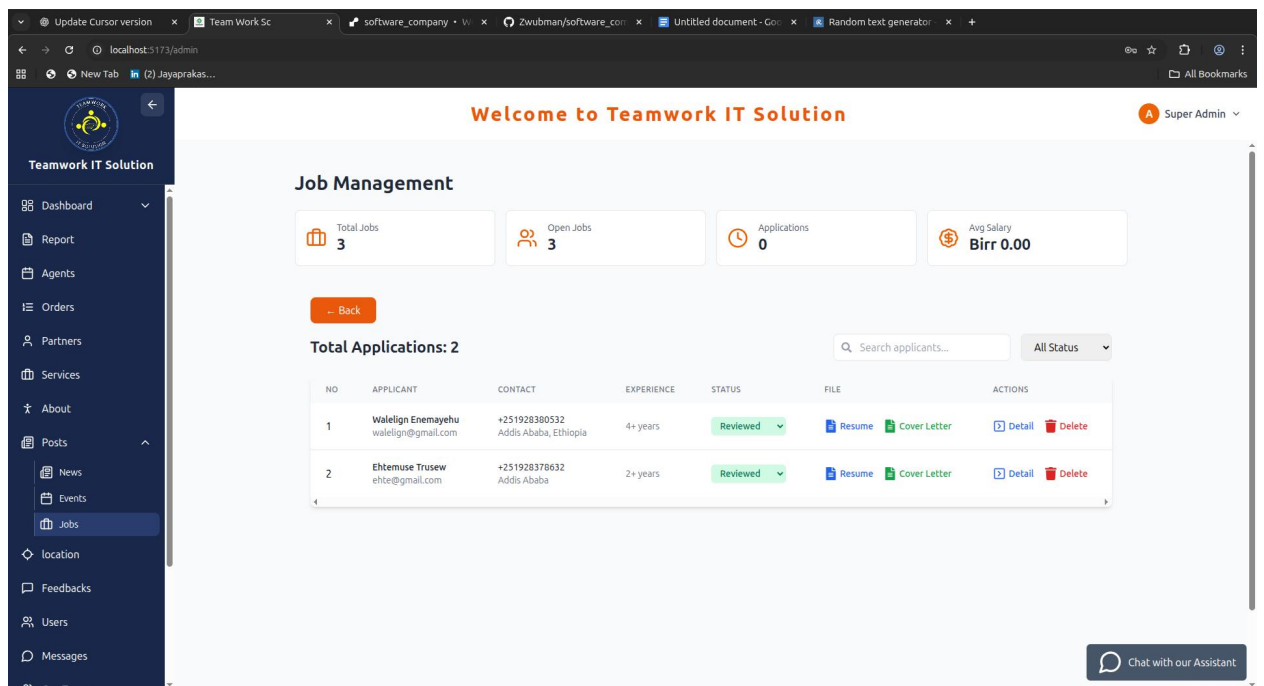


Figure 4.1.1.3 Job application page.

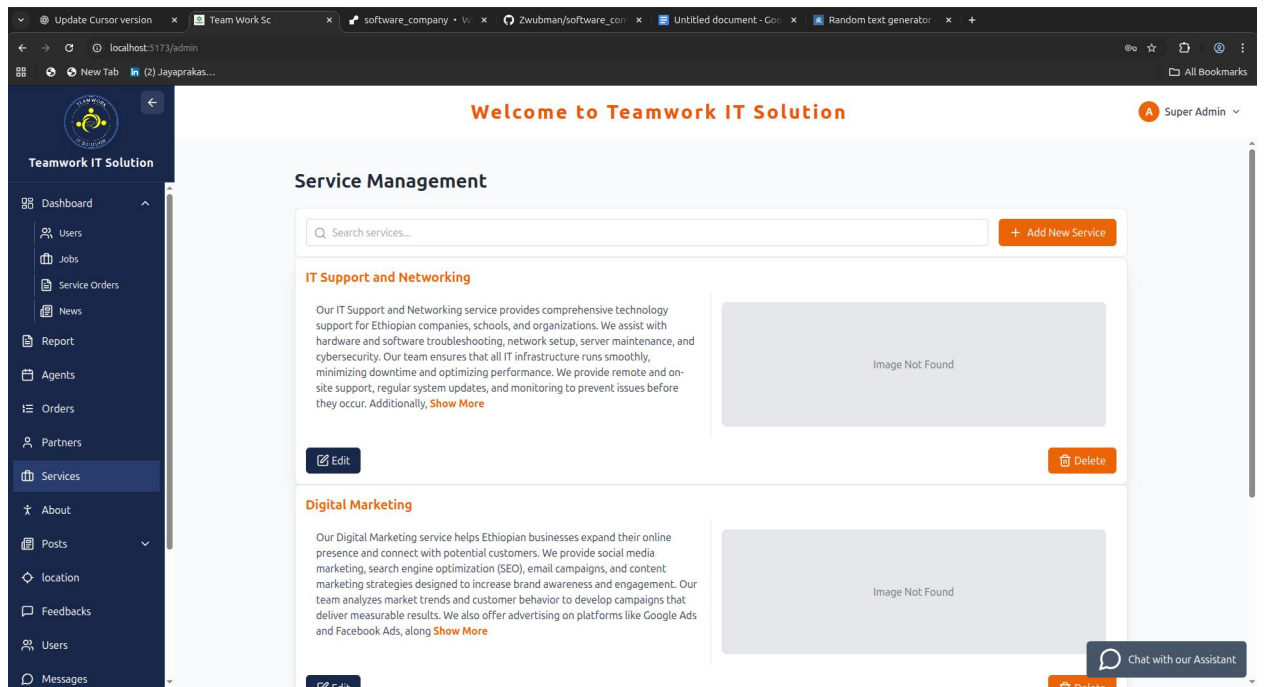


Figure 4.1.1.4 Service page.

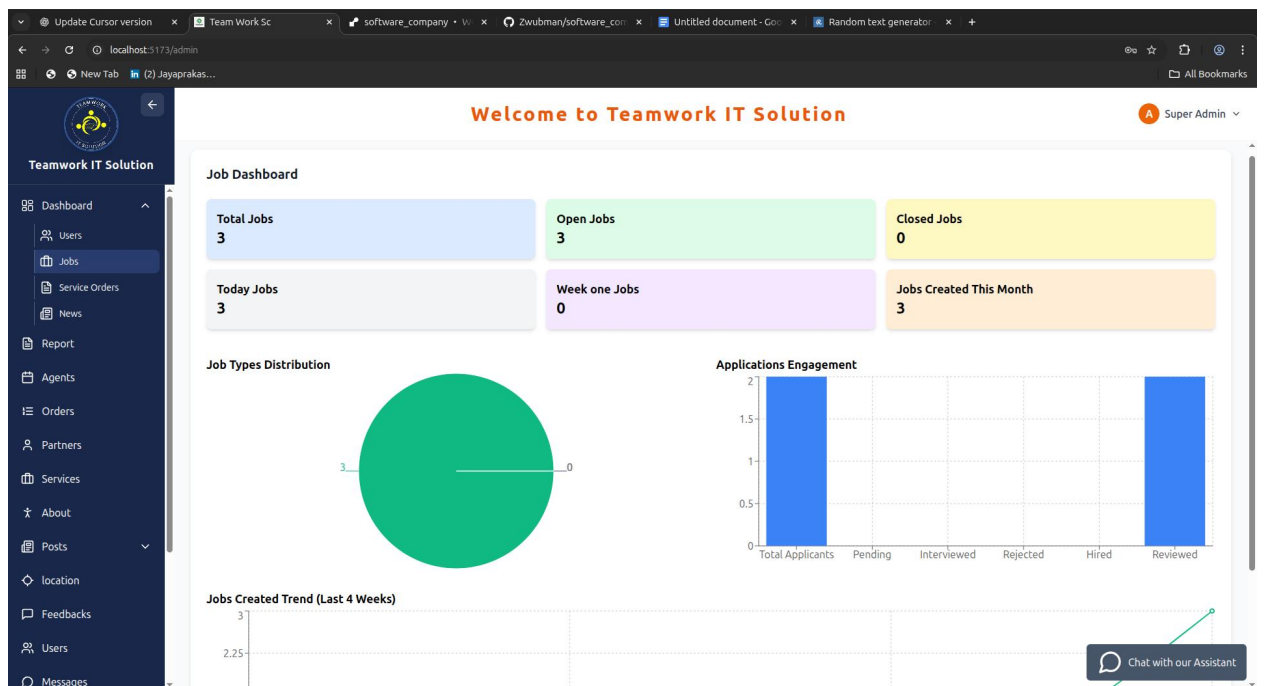


Figure 4.1.1.5 Jobs and applications statistical data page.

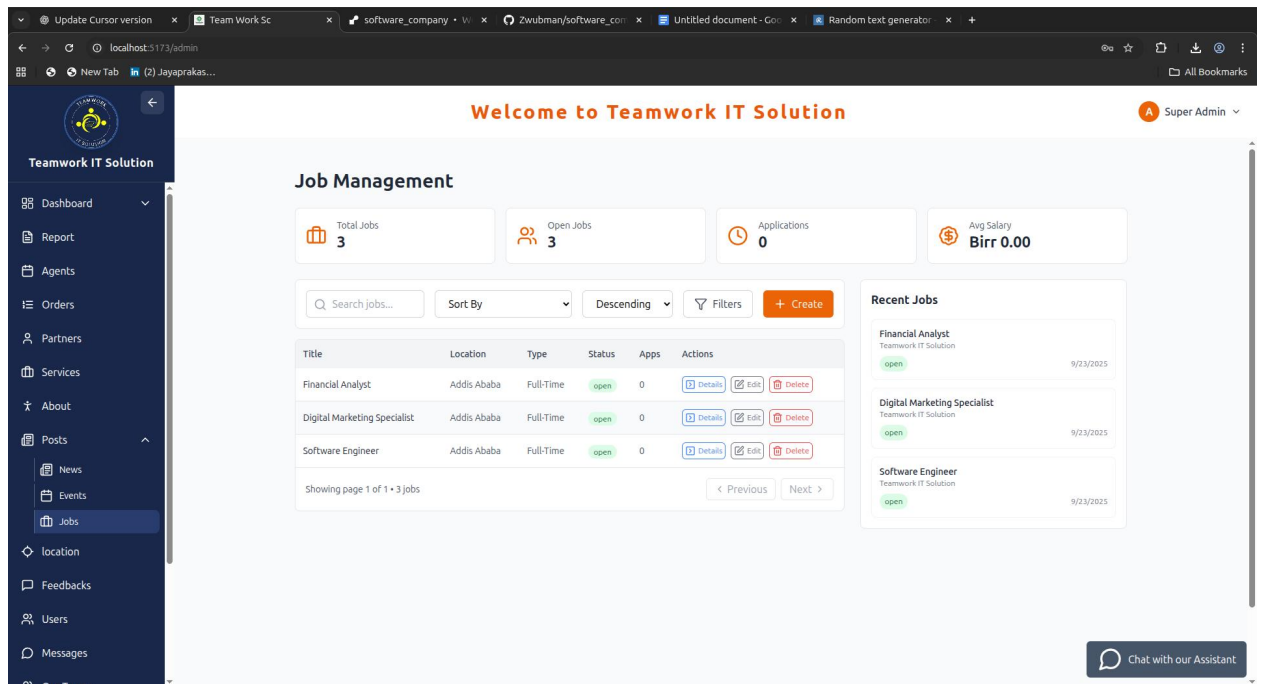


Figure 4.1.1.7 Job management page.

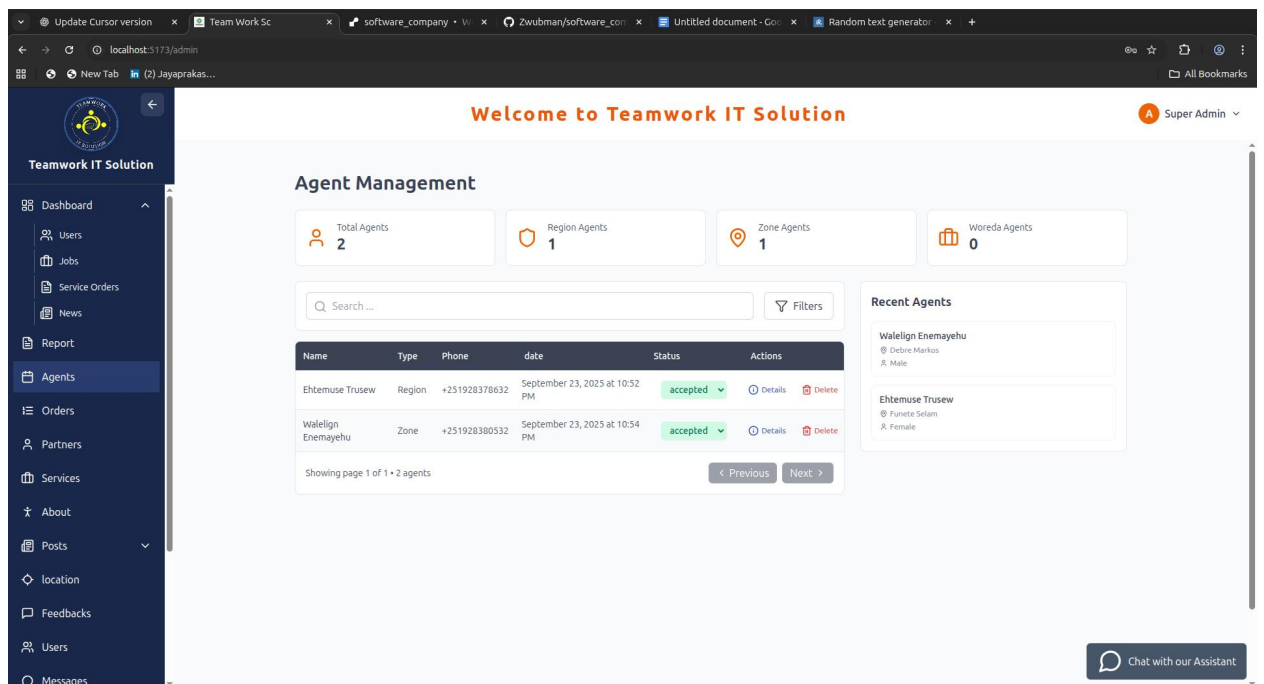


Figure 4.1.1.8 agent requests management page.

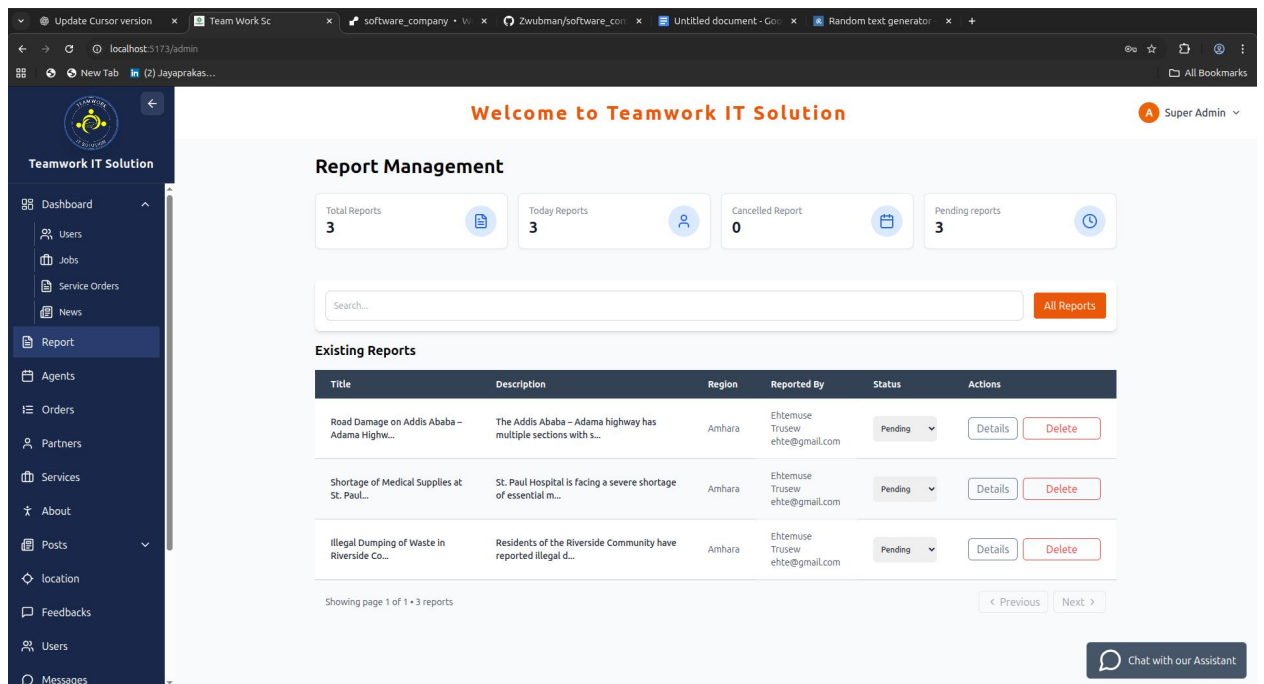


Figure 4.1.1.9 Report management page.

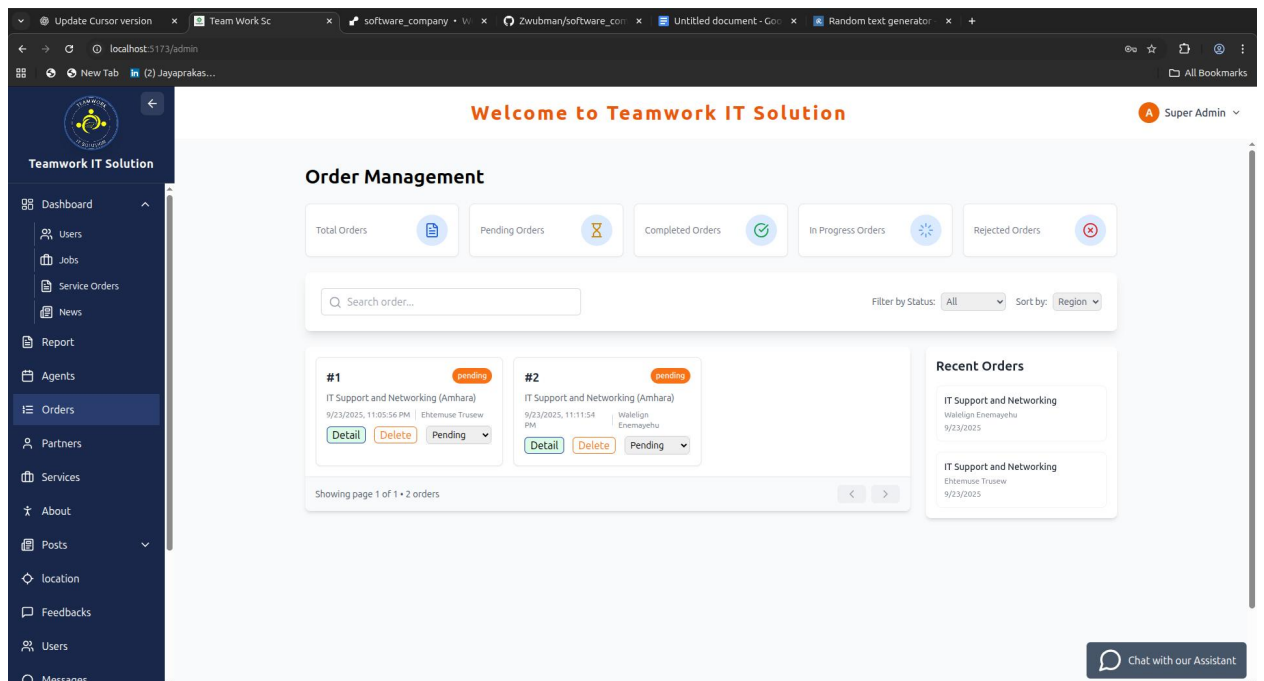


Figure 4.1.1.10 Order management page.

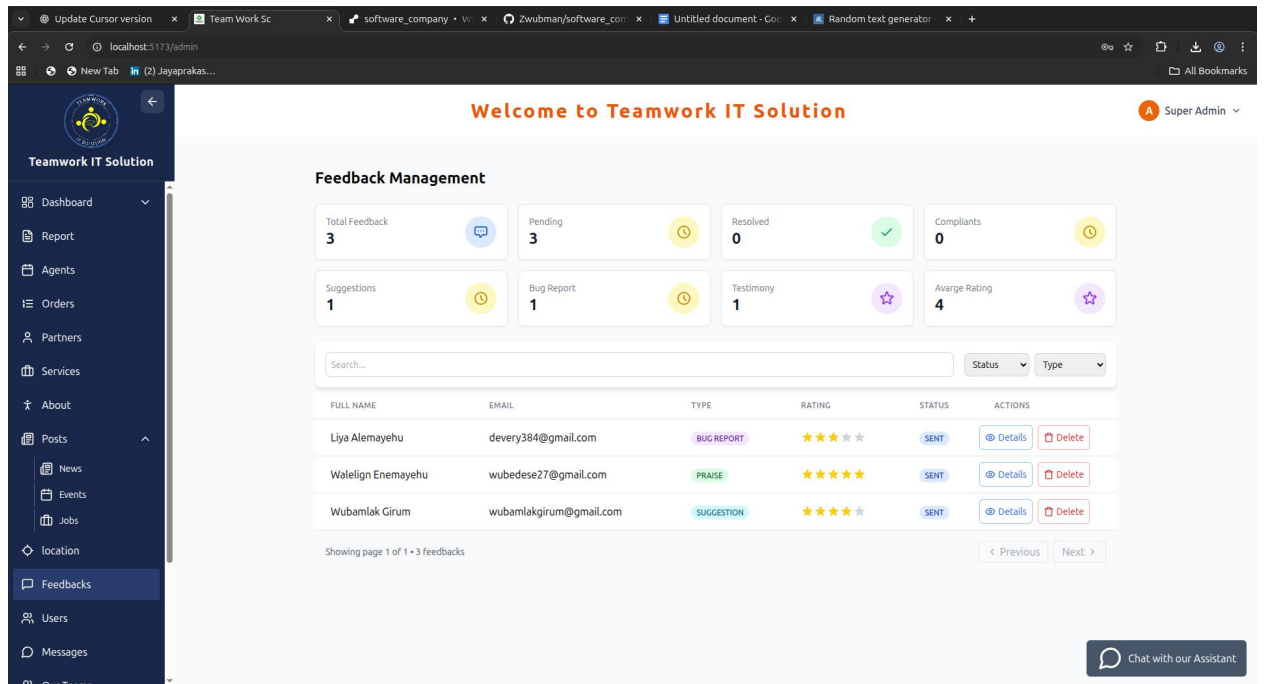


Figure 4.1.1.11 User feedback management page.

Figure 4.1.2 Other side(agent, partner and public side of the website).

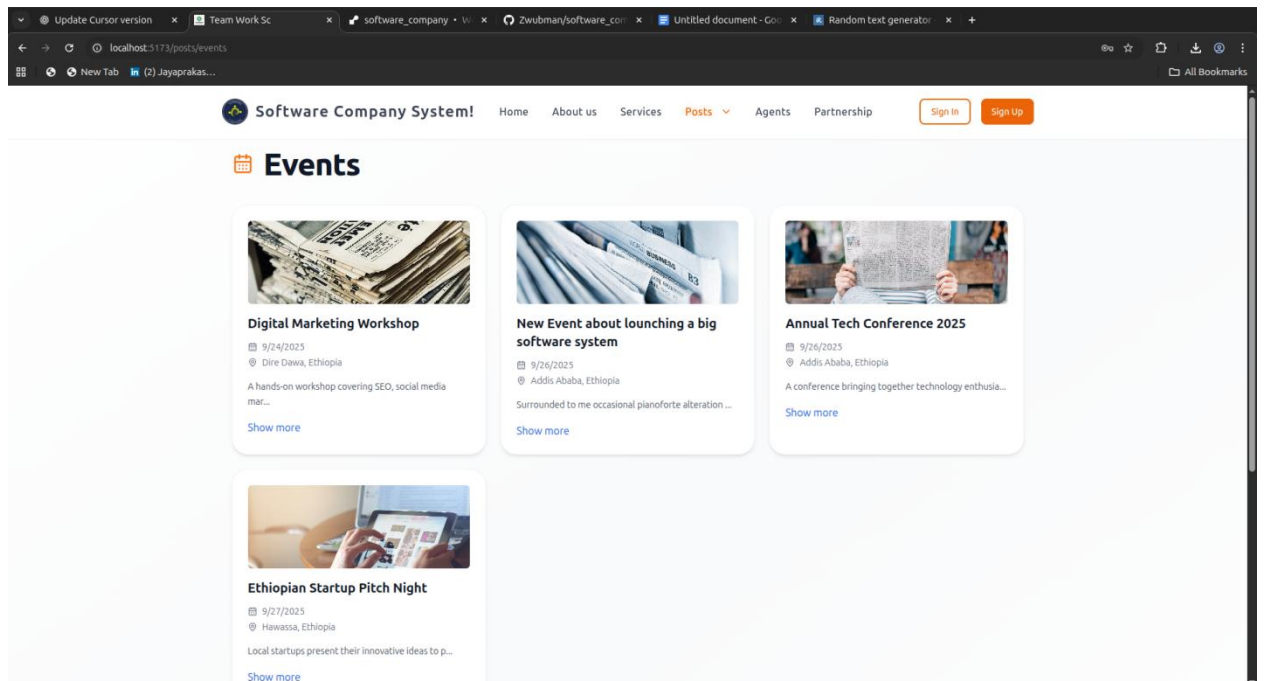


Figure 4.1.2.1 Display event.

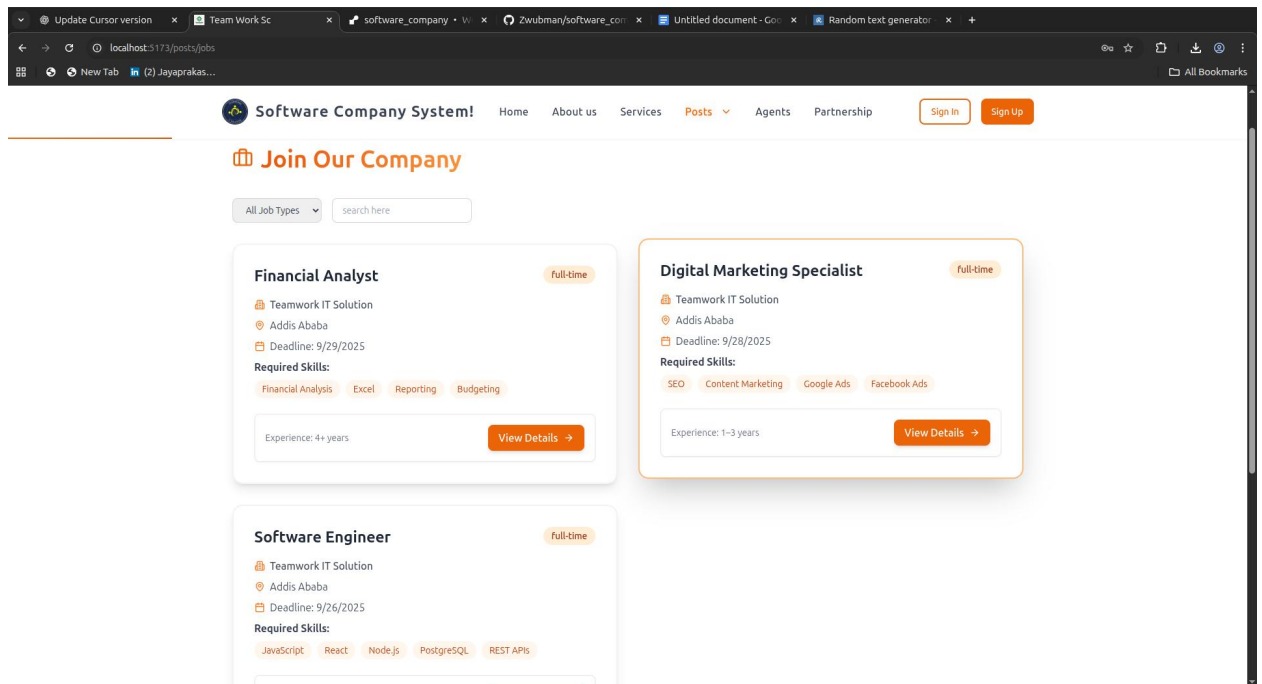


Figure 4.1.2.2 Jobs page.

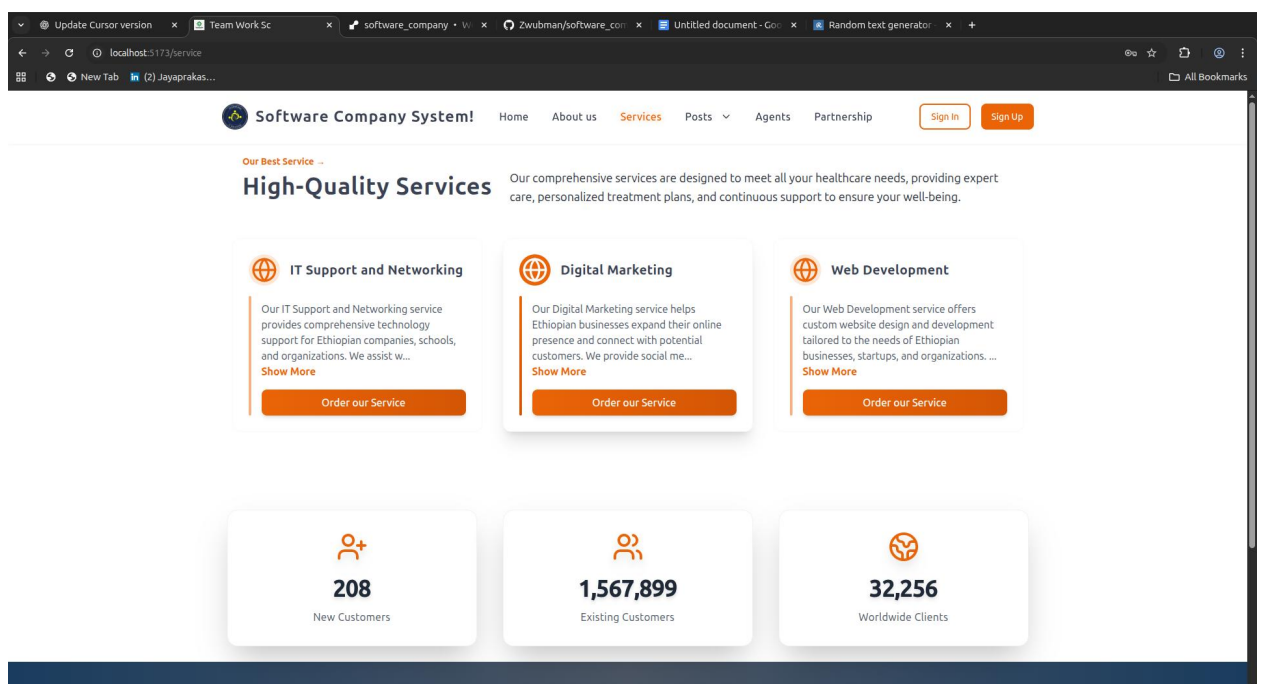


Figure 4.1.2.3 Service page.

4.2 Problem Statement & Justification

Problem Statement

In today's rapidly changing digital environment, software companies are faced with the rising challenge of coping with diverse operations, aligning different stakeholders,

and providing quality services to partners and customers. Traditional organizational structures that are mainly built on manual reporting, compartmentalized communications, and local delivery of services are increasingly proving ineffective in meeting the demands of today's communities. Such outdated modes typically result in delays, inefficiencies, lack of accountability, and partial customer satisfaction.

The primary issue is the absence of an integrated digital platform through which firms are able to manage activities of their system administrators, regional administrators, zonal administrators, woreda administrators, assistants, agents, working partners, and customers in an integrated workflow. Most companies' operations are carried out using fragmented processes, where decision-makers lack access to timely information or monitor the performance of different administrative levels. This is aggravated in businesses that conduct business across regions and districts where coordination between local and central offices often is made difficult by communication breakdowns and delay.

Another issue lies in handling partnerships and collaborations. Contemporary software firms tend to have partnerships with external sources of innovation, technical backup, and capital. Without an organized digital system to handle partnership submissions, collaboration requests, or cooperative endeavors, organizations lose out on excellent opportunities and do not maximize external inputs.

The scalability issue makes the problem even more daunting. While companies can have agents, partners, regional or zonal administrators in hierarchical models in some situations, others could be able to operate with a more straightforward centralized model. Existing solutions do not allow the flexibility of a company in choosing only the features that will fit its organizational model. This lack of flexibility forces companies to either overcomplicate their operations through non-relevant features or not utilize digital tools to their full potential, leaving enormous efficiency and effectiveness gaps.

Justification

The expansion of the Software Company System is strongly justified by the essential necessity of modernizing the way that software companies conduct their day-to-day businesses, interact with their societies, and collaborate with their partners. The rationale for the expansion can be discussed along five significant dimensions: efficiency, accountability, flexibility, customer engagement, and scale.

1. Increasing Efficiency in Operations

Contemporary organizational procedures in the majority of software companies are marred by fragmented communication and time-consuming manual reporting procedures. These processes consume significant resources and time, and managers become mired in redundant details while impairing the ability of the firm to innovate and grow. The Software Company System introduces an automated platform where events, services, operations, and reports are recorded, monitored, and updated in real-time. This eliminates redundancy of effort, reduces errors, and speeds up decision-making procedures.

2. Increasing Accountability and Transparency

In multi-level organizational hierarchies, responsibility gets blurred because reports filter slowly up the hierarchy and reactions lag. This delay in decision-making makes it difficult to hold agents, assistants, or administrators accountable for their actions. The Software Company System is a solution to this issue as it establishes well-defined role-based responsibilities. Each administrator—regional, zonal, or woreda—is responsible for supervising and resolving issues within his or her territory. Any pending matter can be escalated to higher levels through the platform, so that all actions are tracked and recorded.

3. Providing Flexibility for All Software Companies

One of the most compelling arguments in favor of the project is that it is so flexible. Unlike rigid systems that push companies into a single solution for all, the Software

Company System gives each company the option to enable only those features that it desires. If a company has to deal with agents, it can enable agent management; otherwise, this feature does not exist. Likewise, regional, zonal, or woreda organizations can avail themselves of hierarchical administration capabilities, offering small businesses merely the basic features such as posting news, services, jobs, events, and customer feedback management.

4. Enhancing Customer and Community Engagement

Customers are the lifeblood of any successful software company. Without an accessible online interface, however, companies are not able to respond to the growing needs of communities for quick, convenient, and online access to services. The Software Company System comes directly to address this by allowing customers to order services, list job openings, schedule events, list partnership requests, and provide feedback directly through the system.

5. Facilitating Scalability and Future Relevance

The Software Company System is designed to grow along with the company. A small business will start with the core features, and as it expands to serve across more regions or begins to work with third-party partners, the system will easily accommodate those advancements. Scalability allows the platform to be useful in the long term, providing value at every level of company development.

To conclude, the Software Company System is justified as an essential step for every software company wishing to modernize its operations, increase responsibility, boost customer relations, and ensure long-term growth. Its flexibility makes it adaptable to any organizational structure, and its scalability makes it a valuable investment as companies expand. By removing inefficiencies, improving communication, and rendering services readily accessible online, the system is a breakthrough answer for software companies now.

4.3 Objectives of the Project

The objectives of the Software Company System project are guided by the need to adopt a comprehensive, integrated, and flexible digital system that is tailored to the dynamic operations of software firms with differing sizes and organizational structures. The objectives are designed to allow the system to not just address current operational gaps, but also provide a scalable and flexible solution that can adapt to the evolving needs of the firms.

4.3.1 General Objective

The general objective of the project is:

- To design and build a comprehensive digital system that enables software firms to manage their organizational operations, enhance communications and coordination among hierarchical levels, and deliver timely, transparent, and accessible services to their customers and partners.

4.3.2 Specific Objectives

To achieve the above general objective, the project sets out the following specific objectives:

1. Role-Based Access and Management

- To develop a flexible role-based system where software companies can enable or disable modules depending on their company structure.
 - Companies with agents can activate the agent role; otherwise, it can remain dormant.
 - Companies with partners can activate the partner role; otherwise, they can focus on in-house operations.
 - Companies with hierarchical administrators (regional, zonal, woreda) can use these features, while small companies can operate without them.

2. Administrative Oversight and Monitoring

- To provide system administrators with the capacity to manage the entire platform, for instance, capturing news, events, services, and vacancies,

monitoring user activity, viewing reports, and overseeing the delivery of services.

- To enable regional, zonal, and woreda administrators (where applicable) to manage operations within their jurisdiction, solve issues, escalate unresolved issues, and ensure accountability.

3. Enhanced Collaboration and Partnership

- To provide partners a virtual space to propose collaborations, share new ideas, present technological solutions, and offer financial or logistic sponsorship to the company.
- To integrate partner activities into company processes so that shared projects are visible, traceable, and efficiently managed.

4. Agent and Assistant Support

- To allow agents (managers) to report local activities, manage tasks distributed to them, and serve as direct representatives of the company in their areas of operation.
- To equip assistants with the tools to respond to questions, guide customer inquiries, and ensure timely communication between the company and stakeholders.

5. Customer and Community Engagement

- To make the platform functional for customers and users who can:
 - Order services,
 - Apply for jobs,
 - Submit partnership or manager applications,
 - Provide feedback,
 - Take part in organizational events and get detailed information.
- To improve the organization's connection with the community through the digitization of feedback mechanisms and service provision.

6. Service, Event, and Job Management

- To create modules for businesses to support posting and administration of news, events, services, and job openings.
- To facilitate real-time interaction with the community members who can enroll, order, or apply for these events and services.
- To provide administrators with tools to track applications, screen requests for services, and change status for responsiveness and transparency.

7. Data Recording, Tracking, and Reporting

- To provide an internal reporting and analytics system for monitoring:
 - Number of registered users,
 - Number of orders received (per day, per week, per month),
 - Job applications received,
 - Services rendered,
 - Trend in feedback.
- To be able to create real-time reports that inform decision-making and strategic planning for the business.

8. Scalability and Adaptability

- To make sure the system can be adapted to various sizes of organizations—from small businesses with few employees to big organizations with hierarchical administrators and agents.
- To create the system with modular flexibility, i.e., each company can determine which features pertain to its organizational structure and disable others without impacting the overall work process.

4.4 Methodology

We have an Agile development process at Timork IT Solution with emphasis on collaboration, teamwork, and responsiveness through periodic sprints and stand-up meetings daily. We group our project teams according to the different modules of the Software Company System to provide centered, streamlined development and alignment to software company needs.

Every project team consists of programmers, testers, designers, and a project manager who work together to design, develop, test, and deploy the system features. Programmers work hand in hand with designers to translate company requirements—such as role-based access, service management, and reporting—into technical designs and code such that the final product is secure and usable. For the purpose of professional growth, junior developers are also paired with senior engineers so that they can be trained in best practices and improve their technical knowledge in developing the project.

Testers regularly perform progress checks, creating automated and manual tests to find and fix errors at early stages of development. Proactive practices ensure that key functions like service ordering, event posting, feedback submission, and reporting work consistently through the project life cycle.

Team efforts are coordinated, progress monitored, and effective communication facilitated by project managers. Scrum practices are implemented, such as maintaining a project backlog, conducting sprint reviews, and making progress visible to everyone. By integrating teamwork in the Timork IT Solution domain, team members can monitor tasks, update them, and remain accountable at every stage of development.

4.5 Analysis, Results, and Discussion

This subsection discusses the anticipated analysis, results, and discussion of the Software Company System project with the goal of creating a flexible digital platform for software firms. The system was developed despite there being no real user data and live company implementation that limits making a complete evaluation. This section analyzes projected outcomes based on theory models, preliminary estimates, and design objectives of the system and accepts the limitation based on lack of use data.

4.5.1 Analysis

The project team conducted a preliminary analysis based on estimated organizational measures and theoretical estimates before the launch of the system. This included anticipated use of the system, such as anticipated numbers of registered users, number of service requests per cycle, and number of job applications posted through the system. The team further solicited for hypothetical statistics on participation in optional modules, for example, agent reports, partner collaborations, and hierarchical administration (regional, zonal, and woreda levels).

4.5.2 Results

As no real-world user data from actual company operations exist, this section cannot present certain measurable results. However, based on preliminary estimates, the team predicts that the Software Company System will:

- Enable effective communication among company roles for better coordination of system administrators, agents, assistants, partners, and customers.
- Enhance the efficiency of handling service requests, job placements, partnership invitations, and comments.
- Support businesses with optional features that can be switched on or off depending on an organization, giving each business a tailored experience.
- Provide administrators with real-time reporting and tracking, leading to more transparent and responsible operations.
- Foster higher levels of community engagement by allowing users and customers to ask questions, provide feedback, and attend events.

4.5.3 Discussion

Based on the given situation, the conversation relies upon the implications of anticipated results rather than actual findings. The anticipated results suggest that the Software Company System could significantly simplify operation management for software companies by bringing activities such as reporting, service tracking, job management, and customer interaction together under one umbrella.

In general, the talk, findings, and analysis of the Software Company System project emphasize the limitations of the lack of live company data. While forecasted assumptions depict the way the system can make things easier, improve collaboration, and improve service delivery, they cannot replace wisdom from actual usage. Pilot deployments, users' feedback, and real-time monitoring will play critical roles in the future to validate the efficiency of the system and enhance its features to target software firms more comprehensively.

4.6 Solution Proposed

To address the issues that software companies face in coordinating operations, communications, and services, we suggest the development of the Software Company System, a flexible and unifying digital platform that will streamline organizational processes without sacrificing accessibility and collaboration.

1. Development of the Software Company System Platform

The first thing will be to design and host the Software Company System platform. The platform will serve as a central location where administrators, agents, partners, assistants, and clients can easily interact. The system will be simple with an easy-to-use interface whereby users will be able to navigate across different modules such as reporting, service management, job posts, and feedback collection with ease.

Companies can choose which modules to enable based on their specific needs, offering a customized experience.

2. Role-Based and Modular Functionality

The site will be role-based to provide access to various members of the company such as system administrators, regional/zonal/woreda administrators, agents, assistants, partners, and customers. There will be separate functionalities for each role such as monitoring operations, report submission, service requests, or posting events/news. Add-on functionalities can be enabled by firms depending on their structure, e.g., a firm without agents or partners can exclude those modules while firms with hierarchical management can have complete utilization of regional, zonal, and woreda functionalities. This approach makes the system scalable and flexible to any software enterprise.

3. Streamlined Operations and Communication

The site will also simplify workflows by providing features of centralized tracking and management. Functions such as service requests, job applications, event management, partnership proposals, and customer inquiries will be simplified and monitored in real-time. The alert and report management capabilities will allow managers and administrators to respond promptly, improve accountability, and simplify operations. The integrated communication system will offer a platform to unite all the stakeholders and keep them informed regardless of their position or location.

4. Customer/ User Feedback and Participation

To optimize system effectiveness, the platform will also include facilities for collecting customer and user feedback. Users will be in a position to post questions, comments, and suggestions, while businesses can process feedback with a view to increasing the quality of services. Continuous observation of user engagement will provide insight on how the system is performing and areas for improvement, such that the platform is always aimed towards company objectives and community demands.

4.7 Conclusion and Recommendations

The Software Company System project has been able to come up with an integrated digital platform to be used by software companies to improve the management of operations, communication, and provision of services. Although the system has been successfully established, the absence of real user data from actual company implementations prevents the complete analysis of its effectiveness, useability, and overall satisfaction of multiple stakeholders. While the anticipated benefits—role-based workflows, feature modular utilization, and enhanced reporting—imply strong potential for the platform to facilitate businesses while maximizing collaboration and operational efficiency,

Some major recommendations can be formulated based on the analysis of anticipated outcomes and the overall objectives of the Software Company System.

1. Implement a Data Collection Strategy

In order to improve system performance and evaluate its impact, it is crucial to have an integrated data collection strategy. The strategy should capture user behavior, service calls, feedback submission, and module usage from all company roles. By gathering real-time data, the project team will gain insights into user behavior and tendencies, validate design hypotheses, and find opportunities for feature optimization. This empirical foundation ensures any future enhancement and upgrade will be supported by actual organizational needs rather than hypothetical assumptions.

2. Pilot Deployment

The pilot deployment with a limited set of software companies is crucial to getting actionable feedback on system usability and user experience. Testing the platform in real organizational settings will uncover possible issues, workflow limitations, or feature deficits. An experimental pilot phase allows the project team to correct things before mass deployment, so the platform is compatible with the practical needs of various company structures, e.g., having or not having agents, partners, or hierarchical administration.

3. Enhance Support and Training

Providing solid support and training to system users is the key to reaching maximum participation and productivity. Having a support staff to address questions and issues promptly, in addition to step-by-step user guides and instructions, will allow administrators, agents, assistants, and customers to use the platform productively. Regular training sessions for company staff on best practices and feature utilization will facilitate effective use of the system, which will maximize overall productivity and satisfaction.

4. Ongoing Monitoring and Performance Evaluation

For continuous improvement, performance measures should be well defined and system performance should be continuously monitored and evaluated. Feedback from the users of the company and system reports will also point out what needs to be enhanced and what is the impact of each module. Ongoing performance evaluation will be utilized in planning future system updates so that the platform remains current with changing needs of software firms and customers.

In a nutshell, the Software Company System project holds vast potential for revolutionizing the business of software companies, internal communication within them, and engagement with their communities. With the application of a wide data collection approach, pilot rollouts, awareness creation, providing total support and training, and ongoing monitoring of performance, the platform can be made to reap the maximum impact and user adoption. These recommendations will make the system an effective tool for software companies, maintaining effective control, promoting collaboration, and delivering services more efficiently to customers and communities.

Part Five: General Conclusion and Recommendations

General Conclusion

This internship report highlights a comprehensive learning experience at Askuala Link that effectively bridged the gap between academic knowledge and professional practice. Our work on various projects, which required us to be flexible and adaptable, provided invaluable hands-on experience in full-stack development. We became proficient in a wide array of tools and technologies, from frontend frameworks like Next.js and Flutter to backend languages like Node.js. This exposure not only enhanced our practical skills in areas like API design and rapid prototyping but also deepened our theoretical understanding of architectural patterns and agile methodologies.

Beyond the technical skills, the flexible, project-based environment was crucial for developing our soft skills, including teamwork, communication, and resilience. We learned to handle challenges like frequent framework changes and project rewrites, which instilled in us a proactive and entrepreneurial mindset essential for a career in software engineering. Overall, the internship provided a strong foundation for our future careers, demonstrating that real-world experience is essential for gaining a holistic understanding of the industry.

Recommendations

Based on our experience, we offer the following recommendations to enhance future projects and the internship program:

- **Enhance Project Scoping and Client Communication:** To mitigate the challenges of frequent project rewrites and scope changes, it is crucial to invest more time in initial requirement gathering and setting clear expectations with clients. A well-defined project scope and regular check-ins can reduce the need for significant rework and improve overall project efficiency.
- **Implement Structured Training for New Technologies:** Given the fast-paced nature of the work, which often required us to learn new frameworks under tight deadlines, providing structured onboarding or a mentorship program for new technologies would be highly beneficial. This would help new developers become proficient more quickly and reduce the time spent on self-learning.
- **Promote Foundational Knowledge:** While learning new frameworks is important, a strong focus on core foundational concepts should be maintained. Understanding underlying architectural patterns and design principles allows

developers to adapt more easily to new tools and solve problems more effectively, especially when debugging across different technology stacks.

- **Foster Collaborative Problem-Solving:** Continue to encourage a collaborative environment where team members work together to solve difficult problems, rather than working in isolation. This not only helps resolve complex bugs but also strengthens team bonds and facilitates knowledge sharing.
- **Develop Entrepreneurial Thinking:** Encourage interns to think beyond their immediate tasks and look for opportunities for innovation and improvement. Exposure to project planning and technical decision-making at an early stage can help cultivate a proactive, entrepreneurial mindset that is highly valued in the industry.

References

- Askuala Link. Official Website. Retrieved from: <https://www.askualalink.com/>

Appendix

This appendix provides references to useful resources on the software development tools, frameworks, and methodologies relevant to your internship at Askuala Link.

Software Development Tools & Technologies

Next.js

- **Link:** <https://nextjs.org/>
- **Description:** A popular React framework for building fast and scalable web applications. It is ideal for developing server-rendered and static websites.

Flutter

- **Link:** <https://flutter.dev/>
- **Description:** An open-source UI software development kit created by Google. It's used for building natively compiled applications for mobile, web, and desktop from a single codebase.

Node.js

- **Link:** <https://nodejs.org/>
- **Description:** A cross-platform JavaScript runtime environment used for building fast, scalable backend services and APIs, which often powers the "server-side" logic of web applications.

Git

- **Link:** <https://git-scm.com/>
- **Description:** A free and open-source distributed version control system. It allows teams to track changes in source code and coordinate work among multiple developers.

Figma

- **Link:** <https://www.figma.com/>

